Smart Library Project

The Scenario

A university library system. Books are stored in a database (BST) for fast searching by ISBN. When a student borrows a book, it's added to their "Borrowing History" (Stack), so they can see their most recent activity first.

Goal: To understand how different data structures serve different "access patterns" (LIFO for history vs. Logarithmic for searching).

Individual Tasks (5 Members)

1. Catalogue Architect: Build a BST to store book titles and authors indexed by ISBN.
2. Borrowing History: Implement a Stack to keep track of books checked out (Most recent on top).
3. Record Finder: Implement a recursive Search function within the BST to find books by ISBN.
4. ADT Designer: Create the Interface for the Library System to ensure "Information Hiding."
5. Admin Logic: Handle the borrowing/returning process (removing from the catalogue and pushing to the stack).

For this project, students are required to deliver a **Functional Console Interface** that allows a user (Librarian or Student) to interact with the system without modifying the source code.

Requirement: The Console Interface

The interface must provide a menu-driven experience including:

1. **Add Book:** Input ISBN, Title, and Author.
2. **Search Book:** Find details by ISBN ($O(\log n)$ efficiency).
3. **Borrow Book:** Move record to history stack.
4. **View History:** Display all borrowed books (LIFO order).
5. **Exit:** Terminate the program.

Below is the example for the code implementation.

```
import java.util.*;

// ========================================
// THE INTERFACE & ENTITY
// ========================================
interface LibraryADT {
    void addBook(int isbn, String title, String author);
    void borrowBook(int isbn);
    void viewLatestHistory();
    void searchBook(int isbn);
}

class Book {
    int isbn;
    String title, author;
    Book left, right;

    public Book(int isbn, String title, String author) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
    }
}
```

```
// ========================================
// BST & STACK LOGIC
// ========================================
class BookBST {
    private Book root;

    public void insert(int isbn, String t, String a) {
        root = ins(root, isbn, t, a);
    }

    private Book ins(Book r, int i, String t, String a) {
        if (r == null) return new Book(i, t, a);
        if (i < r.isbn) {
            r.left = ins(r.left, i, t, a);
        } else if (i > r.isbn) {
            r.right = ins(r.right, i, t, a);
        }
        return r;
    }

    public Book search(int i) {
        return sea(root, i);
    }

    private Book sea(Book r, int i) {
        if (r == null || r.isbn == i) return r;
        return (i < r.isbn) ? sea(r.left, i) : sea(r.right, i);
    }
}

class BorrowStack {
    private Stack<Book> stack = new Stack<>();

    public void push(Book b) {
        stack.push(b);
    }

    public void show() {
        if (stack.isEmpty()) {
            System.out.println("History is empty.");
        } else {
            // Reverse iteration to show most recent first
            for (int i = stack.size() - 1; i >= 0; i--) {
                Book b = stack.get(i);
                System.out.println("[ISBN: " + b.isbn + "] " + b.title);
            }
        }
    }
}
```

```java
// ========================================
// REQUIREMENT A: FUNCTIONAL CONSOLE INTERFACE
// ========================================
class SmartLibrary implements LibraryADT {
    private BookBST catalogue = new BookBST();
    private BorrowStack history = new BorrowStack();

    public void addBook(int i, String t, String a) {
        catalogue.insert(i, t, a);
    }

    public void searchBook(int i) {
        Book b = catalogue.search(i);
        if (b != null) {
            System.out.println("Found: " + b.title);
        } else {
            System.out.println("Not Found.");
        }
    }

    public void borrowBook(int i) {
        Book b = catalogue.search(i);
        if (b != null) {
            history.push(b);
            System.out.println("Borrowed " + b.title);
        } else {
            System.out.println("Book not in catalogue.");
        }
    }

    public void viewLatestHistory() {
        history.show();
    }

    public void runMenu() {
        Scanner sc = new Scanner(System.in);
        while (true) {
            printMenu();
            System.out.print("Choice: ");
            int choice = sc.nextInt();
            if (choice == 5) break;

            handleChoice(choice, sc);
        }
        sc.close();
    }

    private void printMenu() {
        System.out.println("\n--- SmartLibrary Menu ---");
        System.out.println("1. Add Book");
        System.out.println("2. Search (BST)");
        System.out.println("3. Borrow (Stack)");
        System.out.println("4. History");
        System.out.println("5. Exit");
    }
```

```
    private void handleChoice(int choice, Scanner sc) {
        switch (choice) {
            case 1:
                System.out.print("Enter ISBN: ");
                int i = sc.nextInt();
                System.out.print("Enter Title: ");
                String t = sc.next();
                System.out.print("Enter Author: ");
                String a = sc.next();
                addBook(i, t, a);
                break;
            case 2:
                System.out.print("Enter ISBN to search: ");
                searchBook(sc.nextInt());
                break;
            case 3:
                System.out.print("Enter ISBN to borrow: ");
                borrowBook(sc.nextInt());
                break;
            case 4:
                viewLatestHistory();
                break;
            default:
                System.out.println("Invalid option.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        new SmartLibrary().runMenu();
    }
}
```

Marking Rubric

Criteria	Updated Focus	Weight
Interface Functionality	The console menu runs without crashing and handles invalid inputs gracefully.	20%
BST Search Logic	Recursive search correctly traverses the tree (O(logn) complexity).	20%
Stack History	Borrowing correctly pushes to stack; history displays in LIFO order.	20%
Information Hiding	The Library system uses the ADT Interface; internal structures are private.	20%
Input Validation	System handles cases like non-integer ISBNs or empty search results.	20%